MEETUP + HANDS-ON WORKSHOP

Building AI Agents on Elastic

Tools, Skills, and Workflows

First 30 minutes: how agents actually work. Then we build one together.

Arpit Tripathi

What we'll cover

AGENDA

We start with the four words you need, then look at each one closely, then see them in a real product.

1

How an agent is put together

The four parts of every agent — the agent itself, its tools, its skills, its workflows — and the loop it runs.

8 min

2

Each part, one at a time

What each one is, what a real one looks like, and the rules that keep it from going wrong.

14 min

3

Doing it well, in Elastic

The practices that keep an agent useful, and how Elastic Agent Builder writes down each of the four parts.

8 min

By the end you should be able to: take any agent — one you build, one a vendor demos — and point at its tools, its skills and its workflows. When an agent behaves badly, one of those three is usually the reason.

Finding the answer is not the same as doing the work

One small job, handled the old way and the new way.

The job: a customer writes in saying their invoice looks wrong. Somebody has to check it and reply.

THE OLD WAY — SEARCH

You do every step

- You search for the invoice number
- You open three systems to compare figures
- You work out that a discount was applied twice
- You write the reply and send it

The system handed you information. That was all.

WITH AN AGENT

It does the steps, you check the result

- It looks the invoice up itself
- It queries the three systems and compares them
- It spots the double discount and says which rule caused it
- It drafts the reply and waits for you to approve

You are still in charge — you approve, not dig.

That is the whole idea. An agent is a language model that has been given things it can do, and permission to keep going until the job is finished.

The hard part is not the model. It is deciding what the agent is allowed to do, what it needs to know, and which steps it is not allowed to change.

Three things get called “an AI agent”

Same job — a customer's card payment failed last night — handled three different ways.

A CHATBOT

It answers your question.

You ask “what does payment error 402 mean?” and it explains. Nothing in your systems changes. You still have to go and fix the payment yourself.

Goes wrong by: sounding confident and being wrong.

AN AUTOMATION

It runs your steps.

A script you wrote retries the payment three times, an hour apart. If the real problem is an expired card, it retries three times and fails three times.

Goes wrong by: following the script when the script no longer fits.

AN AGENT

It works out what to do.

It reads the error, sees the card expired, checks whether the customer has a second card saved, charges that one — and if there isn't one, emails the customer and stops.

Goes wrong by: choosing a poor path, and spending money doing it.

You need all three in a real system. Some steps should never change, some questions just need an answer, and a few decisions genuinely need judgement.

Use an agent when the right next step depends on what the last step returned. If it never does, write an automation — it will be cheaper, faster and easier to fix.

What an agent actually does, step by step

Every agent runs this same five-step loop. Right-hand column: the failed payment from the last slide.

1 GOAL	You tell it what you want, in plain words.	*"Recover last night's failed payment for this customer, or tell them why we can't."*
2 THINK	It picks the one next action that gets it closer.	*"I don't know why it failed yet. First look up the payment record."*
3 ACT	It calls one tool, with real values filled in.	*get_payment(id = 88213)*
4 LOOK	It reads what came back — often not what it expected.	*Comes back: declined, reason "card_expired", card ending 4471.*
5 DECIDE	Finished? Try something else? Ask a person?	*"Not finished. Check for another saved card." → back to step 2.*

Step 5 is the one people forget. Without a clear finish line an agent doesn't stop — it loops until the budget runs out, or decides it's done when it isn't.

The four parts of every agent

Think of a new colleague starting on Monday. They need exactly the same four things.

THE AGENT — decides what to do next

The colleague themselves. They read the situation and choose the next step.

Without it: you just have a script.

TOOLS — the things it can do

The apps on their laptop. Email, the finance system, the ticket queue.

Without them: it can give advice, but it can't act.

SKILLS — how your company does it

The handbook you hand them on day one. How reports are written, how refunds are approved.

Without them: it acts, but not the way you do things here.

WORKFLOWS — the steps that never change

The month-end checklist. Same steps, same order, every single month.

Without them: it improvises the one job you needed done identically.

One line to remember: tools are what it CAN DO · skills are what it KNOWS · workflows are what it ALWAYS DOES, in order · the agent DECIDES which.

When an agent disappoints, start by asking which of these four is missing. In our experience it is almost never the model that was the problem.

Tools — the things an agent can actually do

A tool is one action with one clear result. Here is exactly what you write when you make one.

WHAT YOU FILL IN WHEN YOU CREATE A TOOL

Name	get_customer_orders
What it does	“Returns the orders one customer placed between two dates. Newest first. Returns at most 200 orders.”
What it needs	customer_id (text) · from_date · to_date
What it gives back	order number, date, amount, status

The middle line matters most — that sentence is how the model decides to use it.

OTHER TOOLS YOU'D RECOGNISE

- **Spreadsheet:** read a sheet, or write a new one
- **Email:** send one message to one named person
- **Calendar:** list who is free on Thursday morning
- **Search:** find last week's failed orders
- **Ticket:** raise an issue with a title and a priority

Not a tool: “handle the customer request”. It has no clear result and no boundary, so the model cannot tell when it applies — it will either use it for everything or never use it.

Rule of thumb: if you can't write one plain sentence saying what a tool gives back and when not to use it, the model won't be able to use it properly either.

Four rules for building tools

Nearly every "the agent keeps picking the wrong tool" complaint comes back to one of these four.

1	**Keep each tool small** One tool with a switch is really four tools in a trench coat. Split them and the model stops guessing.	INSTEAD OF manage_orders(action = create / update / cancel / refund)	DO THIS **create_order · cancel_order · refund_order**
2	**Write the description for the model** Saying what a tool is NOT for prevents most wrong calls.	INSTEAD OF "Gets order data."	DO THIS **"Returns one customer's orders between two dates, max 200. Not for refunds — use refund_order."**
3	**Give back a little, not a lot** Everything you return fills up the agent's memory and leaves less room for it to think.	INSTEAD OF Returns all 40,000 matching rows	DO THIS **Returns the top 20 rows, plus the total count**
4	**Fix the query when you already know it** Save the open-ended option for questions you genuinely cannot predict.	INSTEAD OF "Write whatever query you think is right"	DO THIS **A ready-made query where the agent only fills in the dates**

And one more: when a tool fails, make it say why. "No rows found for that date range" is far more useful than an empty result.

Skills — the know-how you hand the agent

A skill doesn't do anything. It changes how the agent uses the tools it already has.

A REAL SKILL, MORE OR LESS IN FULL

“Writing the monthly sales report”

- Open with revenue against target, in one sentence.
- Always show this month next to the same month last year.
- If any region is down more than 10%, explain why before showing any table.
- Round money to whole rupees. Never use more than one decimal.
- Close with the three actions we agreed, and who owns each one.

Plain English. The person who knows the job can write it — no engineer needed.

OTHER SKILLS WORTH WRITING

- **Refund policy:** when we refund, when we escalate, what we never do
- **Coding standard:** the rules to follow whenever the agent writes UI code
- **Incident playbook:** how to grade severity and who gets called
- **Customer tone:** how a reply to an angry customer should read
- **Known issues:** problems we already looked into, and what we found

How to tell them apart: if it DOES something, it's a tool. If it tells the agent HOW to do something, it's a skill.

Skills are how last month's lesson becomes next month's default — every rule you write down is one the agent no longer has to work out for itself.

Prompt, skill or tool — where does each thing go?

Three places to put knowledge. Putting things in the wrong one is what makes agents vague and expensive.

	SYSTEM PROMPT	SKILL	TOOL
What it holds	Who the agent is and the rules that always apply	How to handle one kind of job	One action that changes or fetches something
When it is used	Every single time the agent runs	Only when that kind of job comes up	Only when the agent chooses to call it
Written as	A few short paragraphs	A page of plain instructions	A definition: name, description, inputs
Example	“You are our billing assistant. Never promise a refund.”	“How we word a refund email, and when to escalate.”	send_email(to, subject, body)
Watch out for	It grows and grows until the agent ignores all of it	Too vague to act on, or so long nobody maintains it	Doing two jobs in one tool

The most common mistake: everything goes in the system prompt. It gets longer every week, the instructions start competing, and the agent stops following any of them.

A good habit: when you're about to add a paragraph to the prompt, ask whether it's really a skill — needed for one job, not for every single run.

Workflows — the steps you don't let it change

You write the steps and the order once. It runs the same way every time, and you can read it afterwards.

A WORKFLOW, WRITTEN OUT

Nightly sales digest — runs at 05:30, every day

1. Pull yesterday's orders from the database
2. Check the totals against the ledger. If they disagree, stop and alert finance.
3. Build the two charts
4. Ask the agent to write the summary, using the report skill
5. Send at 06:00 to the leadership list

Only step 4 uses the model. Steps 1, 2, 3 and 5 are the same every night.

OTHER THINGS WORTH PINNING DOWN

- **New joiner setup:** create account → order laptop → enrol in training → tell the manager
- **Alert response:** add the logs → grade it → wake someone if it's severity 1 or 2
- **Release:** run tests → security scan → a person approves → deploy
- **Refund over ₹50,000:** check the order → check the policy → a manager approves → pay

Not a workflow: "answer any question about our data". That is exactly where you want the agent choosing — pinning it would mean writing a branch for every question anyone might ask.

This is also where human approval belongs. "A person signs off before money moves" is a step in a list — not something you hope the model remembers to do.

How much do you decide in advance?

The same job can be built three ways. The only thing that changes is how many of the decisions you made yourself, before the agent ever ran.

	YOU DECIDE EVERY STEP	YOU DECIDE THE SHAPE, IT FILLS ONE STEP	THE AGENT DECIDES
The job: a customer asks for a refund	Look up the order → check it is under 30 days → check the amount → refund → email them. The agent only fills in the order number.	The same five steps — but step 2 is left open: the agent decides whether this refund is fair, given what the customer wrote.	"Sort out this complaint." The agent chooses what to look at, and whether to refund, ask for more detail, or pass it to a person.
What the model is allowed to do	Fill in values inside your steps. Nothing else.	Make one judgement, inside the fence you drew.	Pick the tools, the order, and when to stop.
Same request tomorrow — same result?	Yes. Always.	Mostly. The open step can vary.	Often not. It may take a different route.
Cost and speed	Cheapest and fastest.	In the middle.	Most model calls, so slowest and dearest.
Best used for	Payments, month-end close, anything audited.	Triage: fixed checks, one call that needs judgement.	Open questions you could not write steps for.
How it goes wrong	Reality doesn't match your steps, so it stops.	The open step wanders off the point.	It picks a poor path and spends time and money on it.

Rule of thumb: decide the steps yourself wherever a mistake is expensive or hard to undo. Leave it to the agent where the question is genuinely open.

All four working together on one ordinary job

The monthly sales report — the thing somebody does by hand on the first of every month.

WORKFLOW
the fixed part

Pull the figures → **Check them** → **Build the charts** → **Write the summary** → **Send at 6am**

TOOLS — what it can do

run_sales_query · write_spreadsheet · make_chart · send_email

Four small things, each giving back one clear result.

SKILL — what it knows

"Writing the monthly sales report": lead with revenue against target, compare to last year, explain any region down more than 10%.

THE AGENT — the part that decides

Step 2 shows revenue is down 18%. An automation would have sent the report anyway. The agent notices the drop, and because the skill says a big fall must be explained first, it calls run_sales_query a second time to break the number down by region, finds that one region lost a large account, and puts that at the top of the email.

The workflow got it sent. The tools did the work. The skill made it useful. The agent noticed the problem.

Best practices — what we'd tell you before you start

None of these cost anything to adopt. Each one saves you about a week.

1 Name the four parts before you build one

Write down, on one page, what it can do, what it needs to know, and which steps must never change. Most stuck agents are a missing skill or a loop with no end — not a weak model.

2 Give every loop a limit

A cap on steps, a cap on spend, and a finish line you can check without reading the whole conversation. Nothing in an agent makes it stop on its own.

3 Keep tools small, and their descriptions sharp

One action, one clear result, and a sentence that says what it will not do. That sentence is what the model reads when it chooses.

4 Give back a little, not a lot

Return the twenty rows that matter, not the twenty thousand you found. Whatever you hand back takes up room the agent needed for thinking.

5 Put approvals in the workflow

At the step that cannot be undone. Never rely on the model deciding to ask — and remember a confirmation prompt in a chat window does not protect an API call.

6 Write down what you learn

Every lesson you turn into a skill is one the next run doesn't have to learn again. That is the whole compounding effect.

And the one that matters most: start smaller than feels useful. One agent, three tools, one written skill, one pinned step — then add only what the failures ask for.

The same four parts in Elastic Agent Builder

Nothing new to learn — the four ideas are the same. This is how Elastic asks you to write each one down.

TOOLS You fill in a form, you don't write code

Elastic ships a catalogue of built-in tools you can't edit (they show a padlock). On top of those you add your own, of four kinds: a ready-made ES|QL query where the agent fills in the blanks · an index it can search freely · a tool from another system over MCP · a call to one of your workflows.

SKILLS Instructions + tools + reference material

An Elastic skill bundles three things: guidance written in Markdown, the tools needed for that job, and any reference documents or runbooks. Built-in ones ship with each solution; you add your own.

WORKFLOWS Steps written in a YAML file

A list of steps kept outside the model. Your agent can call one as a tool, and you can review the file like code.

AGENTS A persona, a prompt, a model, a set of skills

That is all an agent is here. It answers from your Elasticsearch data, and you reach it from chat in Kibana, from an MCP client, from another agent, or over REST.

Two details worth knowing before you build.

Skills only load when they are needed, so you can give an agent many without filling up its memory. And when the agent wants to change something, the chat asks you to confirm first — but calling a tool straight through the API skips that check.

Everything from the first half carries over — only the names change. **Now let's go and build one.**

Thank you

Questions — then let's build one.